

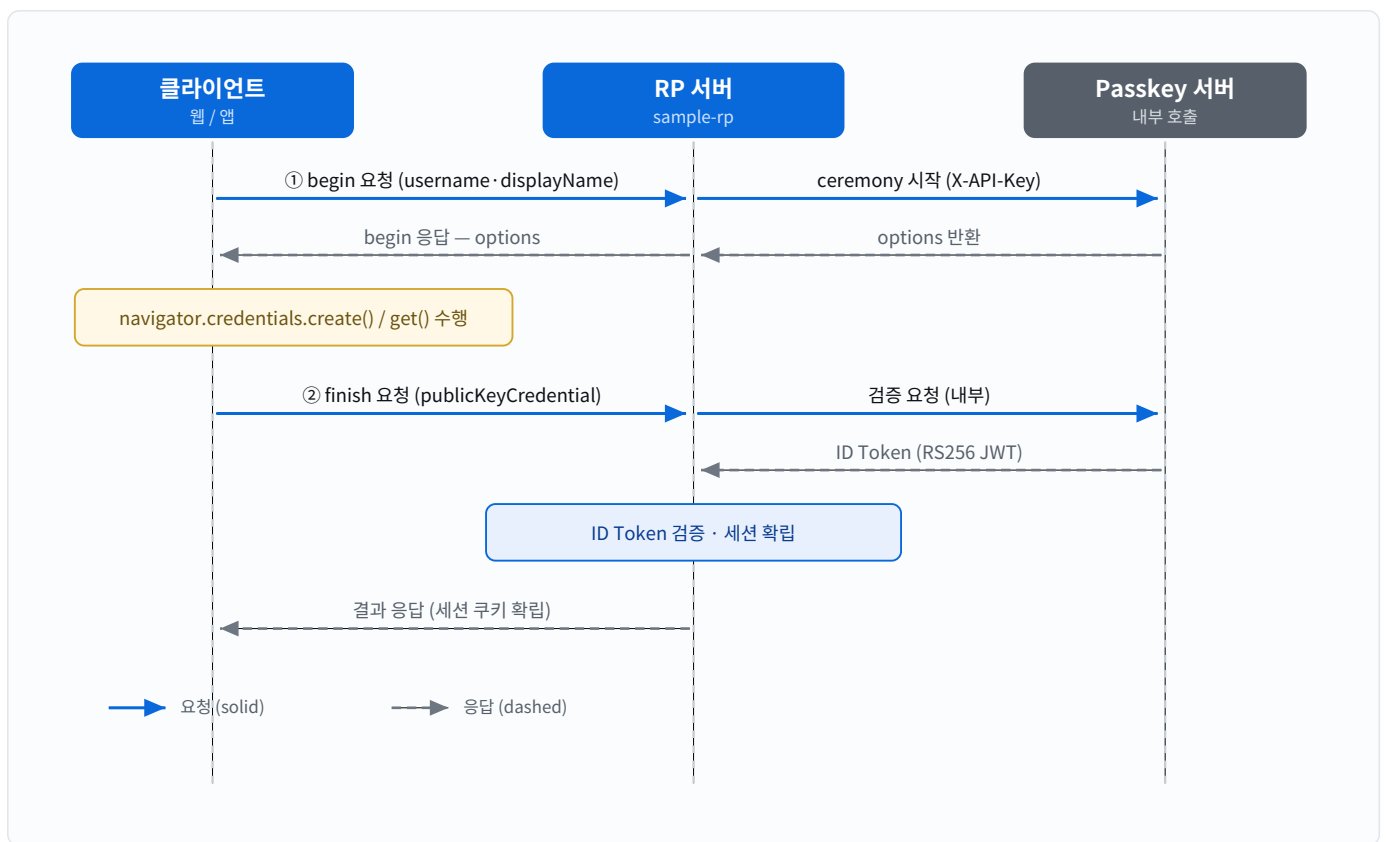
RP 서버 API 명세서 (클라이언트 → RP 서버)

클라이언트(웹/앱)가 **RP 서버**에 패스키 등록·인증을 요청하는 API 명세서입니다. sample-rp의 실제 구현(`/passkey/**`)을 기준으로 작성했습니다.

- **호출 관계**: 클라이언트(브라우저 JS / 모바일 앱) → **RP 서버**(당신이 구현하는 서버). RP 서버는 내부적으로 Passkey 서버를 호출하지만, 그 부분은 클라이언트에 노출되지 않습니다.
- **범위**: 패스키 등록(`/passkey/register/**`)과 인증(`/passkey/authenticate/**`)입니다.
- **기준 주소**: 이 문서의 예시는 RP 서버를 `https://dev-passkey.crosscert.com` 으로 가정합니다(클라이언트가 요청을 보내는 대상). 로컬 개발 시에는 `http://localhost:9090` (sample-rp 기본 포트)으로 바꿔 읽으세요.

1. 개요

RP 서버는 클라이언트와 패스키 ceremony를 주고받습니다. 각 ceremony는 **begin**(challenge 발급)과 **finish**(결과 검증) 두 단계입니다.



용어: 이 문서에서 ceremony의 두 단계를 **begin/finish**로 부릅니다(실제 엔드포인트 이름과 동일: `/passkey/register/begin` · `/finish` 등). begin 응답 안에 담겨 오는 WebAuthn 파라미터 묶음은 **options**(`publicKeyCredentialCreationOptions` / `RequestOptions`)라고 부릅니다.

핵심 사항입니다.

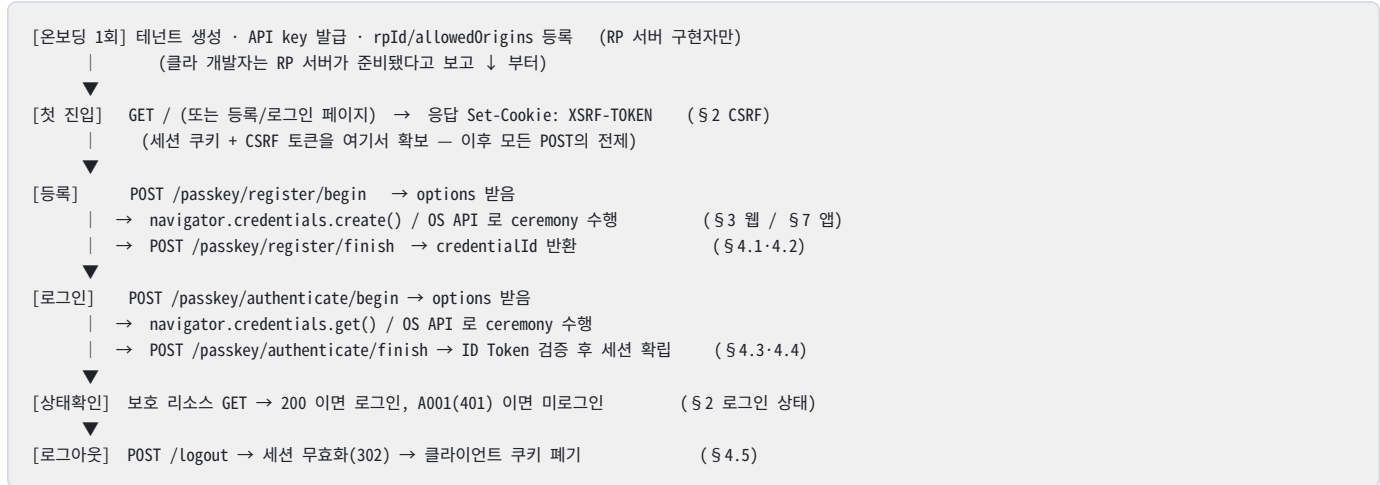
- **userHandle** 은 RP 서버가 내부에서 만듭니다. 클라이언트는 `username` · `displayName` 만 보냅니다. userHandle 생성·Passkey 서버 호출·ID Token 검증은 모두 RP 서버가 처리하며 클라이언트에는 보이지 않습니다.
- **세션 기반입니다**. RP 서버가 `begin` 에서 ceremony 진행 상태를 **세션에 저장**하고(이를 pending token이라 하며 클라이언트에는 노출되지 않습니다) `finish` 에서 같은 세션으로 이어받습니다. 클라이언트는 세션 쿠키(같은 origin이면 `credentials: 'same-origin'`)를 유지해야 합니다.
- **CSRF 보호가 적용됩니다**. 모든 POST 요청에 CSRF 토큰을 헤더(`X-XSRF-TOKEN`)로 보내야 합니다. 토큰을 얻는 법과 선행 단계는 § 2에 있습니다.
- 인증 성공(`authenticate/finish`) 시 RP 서버가 ID Token을 검증한 뒤 **자체 세션**을 확립합니다. 클라이언트는 별도 토큰을 받지 않고 세션 쿠키로 인증됩니다.

- **X-API-Key** 는 클라이언트가 쓰지 않습니다. 이는 RP 서버가 Passkey 서버를 호출할 때만 쓰는 서버-서버 비밀입니다. 클라이언트가 RP 서버에 보내는 인증 요소는 세션 쿠키 + **X-XSRF-TOKEN** 뿐입니다.

읽는 순서 안내 - 앱(iOS/Android) 개발자: § 1 전체 흐름 → § 2 공통 규약 → § 7 모바일 → § 8.2 에러. § 3(웹 base64url 변환)·§ 5(브라우저 JS)·§ 8.1(웹 에러)는 웹 전용이라 건너뛰어도 됩니다. - **웹 개발자:** § 2 → § 3 → § 4 → § 5 → § 8.1. - **RP 서버를 직접 구현하는 경우에만** 아래 "온보딩"·"ID Token 검증"이 필요합니다. 클라이언트(웹/앱) 개발자는 이미 RP 서버가 준비됐다고 보고 § 2로 넘어가세요.

전체 흐름 (한눈에)

처음 통합하는 분은 이 순서로 전체 그림을 잡으세요. 각 단계의 상세는 괄호 안 절을 참조합니다.



시작하기 전 (온보딩) — RP 서버 구현자용

이 절은 RP 서버를 직접 세팅하는 분만 보면 됩니다. 클라이언트(웹/앱) 개발자는 건너뛰세요.

RP 서버를 Passkey 서버에 연결하려면 먼저 다음이 필요합니다.

1. **테넌트 생성** — 관리 콘솔에서 RP를 위한 테넌트를 만듭니다. 이때 **rpId** (RP 서버 도메인)와 **allowedOrigins** (RP 서버 origin 목록)를 등록합니다. 이 값이 패스키의 신뢰 경계가 됩니다.
2. **API key 발급** — 그 테넌트에 **registration** · **authentication** scope를 가진 API key를 발급받습니다. 평문 키는 발급 시 1회만 표시됩니다.
3. **tenantId 확인** — ID Token의 **iss** / **aud** 검증에 쓸 tenantId(UUID 형식)를 받아둡니다.

테넌트 생성·키 발급·도메인 등록의 구체 절차는 운영 환경에 따라 다릅니다(관리 콘솔 또는 운영팀). 로컬/단일 인스턴스 구성은 [single-instance-deployment.md](#) § 4를 참조하세요.

ID Token 검증 — RP 서버 구현자용

이 절도 RP 서버를 직접 구현하는 분만 보면 됩니다. 클라이언트는 ID Token을 직접 다루지 않으며(RP 서버가 검증 후 세션을 확립), **Java라면** 제공되는 **검증 라이브러리**에 아래가 내장되어 있습니다. 클라이언트(웹/앱) 개발자는 건너뛰세요.

authenticate/finish 에서 Passkey 서버가 발급한 ID Token(RS256 JWT)을 RP 서버가 검증할 때 지켜야 할 사항입니다.

- **서명:** JWKS([/.well-known/jwks.json](#))에서 JWT 헤더의 **kid** 에 해당하는 RSA 공개키로 RS256 검증. **alg** 가 **RS256** 인지도 확인(alg confusion 방지).
- **exp** : 만료 검증. 시계 오차를 고려해 **clock skew leeway(예: ±60초)** 를 두는 것을 권장합니다.
- **iss** / **aud** : **iss** = <Passkey 서버 주소>/<tenantId> , **aud** = <tenantId> . tenantId 표기 차이로 인한 검증 실패는 § 6 P004를 참조하세요.
- **JWKS 캐싱:** 매 요청마다 JWKS를 가져오지 말고 캐시합니다(권장 TTL 수 분). **키 회전 주의** — 서버가 서명키를 교체하면 캐시에 없는 새 **kid** 가 올 수 있습니다. **kid** 를 못 찾으면 캐시를 즉시 무효화하고 1회 재조회하도록 구현하는 것이 안전합니다(TTL만 의존하면 회전 직후 TTL 동안 검증이 실패할 수

있음).

2. 공통 규약

△ **선행 요구사항** — **CSRF 토큰을 먼저 받으세요(모든 클라이언트 공통 첫 단계)**. begin/finish/logout 등 모든 **POST 전에** 아래를 먼저 해야 합니다. 안 하면 모든 POST가 **403 Forbidden** 으로 막힙니다. 1. RP 서버에 **GET 요청 한 번**(예: **GET /** , 또는 등록/로그인 페이지) — 응답의 **Set-Cookie: XSRF-TOKEN=...** 으로 토큰이 내려옵니다. 2. 그 쿠키 값을 읽어 이후 POST의 **X-XSRF-TOKEN** 헤더에 실어 보냅니다. 서버 렌더링 페이지(브라우저)는 페이지를 GET으로 여는 순간 ①이 자동으로 일어나므로 추가 작업이 없지만, **SPA·모바일 앱은 첫 POST 전에 GET 1회를 직접 넣어야** 합니다.

인증 / 세션 / CSRF

- **세션 쿠키**: ceremony는 **begin** → **finish** 가 같은 세션이어야 합니다. **RP 서버와 클라이언트가 같은 origin**이면 fetch에 **credentials: 'same-origin'** 을 지정합니다. SPA를 **다른 도메인**에서 호스팅하는 경우(cross-origin)에는 **credentials: 'include'** 를 쓰고, RP 서버가 **Access-Control-Allow-Credentials: true** 와 정확한 **Access-Control-Allow-Origin** (와일드카드 ***** 불가)을 응답해야 쿠키가 오갑니다.

CSRF 토큰을 얻는 방법

토큰은 클라이언트가 만드는 것이 아니라 RP 서버(Spring Security **CookieCsrfTokenRepository**)가 생성해 쿠키 **XSRF-TOKEN** 으로 내려줍니다. 클라이언트는 그 값을 읽어 **헤더 X-XSRF-TOKEN** 에 복사해 되돌려 보낼 뿐입니다(double-submit cookie 패턴). 전용 토큰 발급 API는 없으며, 위 선행 단계의 첫 GET 응답에 묻어 옵니다. 클라이언트 종류별 획득 방법입니다.

클라이언트	토큰 출처	보내는 방법
서버 렌더링 페이지(Thymeleaf)	페이지의 <code><meta name="csrf"></code>	<code>document.querySelector('meta[name=csrf]').content</code> 를 헤더에 실음
SPA / fetch / 모바일 웹뷰	쿠키 XSRF-TOKEN	쿠키 값을 읽어 헤더에 실음
모바일 네이티브 앱	응답의 Set-Cookie: XSRF-TOKEN=...	쿠키 jar에서 값을 꺼내 헤더에 실음(\$ 7)

쿠키는 **HttpOnly** 가 **아니므로** (**withHttpOnlyFalse**) JavaScript에서 읽을 수 있습니다. SPA·모바일 웹뷰에서 쿠키로 토큰을 얻는 예시입니다.

```
// 쿠키 XSRF-TOKEN 값을 읽어 헤더에 실는다 (SPA / 웹뷰용)
function csrfToken() {
  const raw = document.cookie.split('; ')
  .find(c => c.startsWith('XSRF-TOKEN='))?.split('=')[1] ?? '';
  return decodeURIComponent(raw); // 쿠키 값에 인코딩 문자가 섞일 수 있어 방어적으로 디코딩
}

await fetch('/passkey/register/begin', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', 'X-XSRF-TOKEN': csrfToken() },
  body: JSON.stringify({ username, displayName }),
  credentials: 'same-origin' // cross-origin SPA 라면 'include'
});
```

순수 SPA 주의: \$ 5의 **postJson** 헬퍼는 **Thymeleaf `<meta name="csrf">`** 에서 토큰을 읽으므로(sample-rp 전용), 메타 태그가 없는 순수 SPA가 그대로 복붙하면 토큰이 빈 문자열이 되어 **403** 이 납니다. SPA는 위 **csrfToken()** (쿠키 기반)으로 바꿔 쓰세요.

로그아웃 / 로그인 상태

- **로그아웃**: **POST /logout** — 세션을 무효화하고 **/** 로 리다이렉트 (**302**)합니다. CSRF 토큰이 필요합니다. 로그아웃 후 클라이언트는 보관 중인 세션 쿠키를 폐기하세요(\$ 4.5).
- **로그인 상태 확인**: 전용 상태 조회 엔드포인트는 없습니다(sample-rp 기준). 로그인 여부를 알아야 하면 RP 서버에 **인증 시 사용자 정보를 반환하고 미인증 시 A001 (401)**을 주는 엔드포인트를 추가하세요. 클라이언트는 그 엔드포인트를 호출해 **200이면 로그인, 401이면 미로그인**으로 판단합니다.

SPA·앱은 보통 부팅 시 1회 호출해 초기 인증 상태를 잡습니다.

응답 Envelope (`ApiResponse<T>`)

모든 응답은 envelope으로 감싸집니다.

```
// 성공
{
  "success": true,
  "code": "OK",
  "message": "Success",
  "data": { /* 엔드포인트별 응답 */ },
  "error": null,
  "traceId": "a1b2c3...",
  "timestamp": "2026-06-01T12:00:00"
}

// 실패
{
  "success": false,
  "code": "W002", // § 6 ErrorCode
  "message": "No pending registration in session",
  "data": null,
  "error": { "errorCode": "W002", "fieldErrors": null },
  "traceId": "a1b2c3...",
  "timestamp": "2026-06-01T12:00:00"
}
```

아래 각 엔드포인트의 응답은 envelope의 `data` 필드 내용만 표기합니다(웹은 `data` 안의 `challenge` · `id` 를 변환해야 하며 § 3에서 다룹니다).

3. 클라이언트 연동 — base64url ↔ ArrayBuffer 변환 (웹 전용·필독)

이 절은 브라우저 웹 개발자용입니다. iOS는 § 7.2(Swift 확장), Android는 § 7.3(변환 불필요)으로 가세요.

△ `begin` 응답의 options를 `navigator.credentials.create()/get()` 에 그대로 넘기면 **TypeError** 로 실패합니다. WebAuthn JS API는 `challenge` · `user.id` · `excludeCredentials[].id` · `allowCredentials[].id` 를 `ArrayBuffer` 로 요구하는데, 응답은 전부 **base64url** 문자열이기 때문입니다. 반대로 `finish` 에 보낼 때는 브라우저가 돌려준 `ArrayBuffer` 필드들을 다시 **base64url** 문자열로 인코딩해야 합니다.

아래는 sample-rp가 실제로 쓰는 변환 헬퍼입니다(`/js/helpers.js`).

```

// base64url → ArrayBuffer
export function b64urlToBuf(s) {
  const pad = '='.repeat((4 - s.length % 4) % 4);
  const bin = atob((s + pad).replace(/-/g, '+').replace(/_/g, '/'));
  const buf = new Uint8Array(bin.length);
  for (let i = 0; i < bin.length; i++) buf[i] = bin.charCodeAt(i);
  return buf.buffer;
}

// ArrayBuffer → base64url
export function bufToB64url(buf) {
  const bin = String.fromCharCode(...new Uint8Array(buf));
  return btoa(bin).replace(/\+/g, '-').replace(/\//g, '_').replace(/>$/, '');
}

// 등록 options 디코딩 (서버 → 브라우저)
export function decodeCreationOptions(opts) {
  return {
    ...opts,
    challenge: b64urlToBuf(opts.challenge),
    user: { ...opts.user, id: b64urlToBuf(opts.user.id) },
    excludeCredentials: (opts.excludeCredentials || []).map(c => ({ ...c, id: b64urlToBuf(c.id) }));
  };
}

// 등록 결과 인코딩 (브라우저 → 서버)
export function encodeAttestationCredential(cred) {
  return {
    id: cred.id,
    rawId: bufToB64url(cred.rawId),
    type: cred.type,
    response: {
      clientDataJSON: bufToB64url(cred.response.clientDataJSON),
      attestationObject: bufToB64url(cred.response.attestationObject)
    },
    clientExtensionResults: cred.getClientExtensionResults()
  };
}

// 로그인 options 디코딩
export function decodeRequestOptions(opts) {
  return {
    ...opts,
    challenge: b64urlToBuf(opts.challenge),
    allowCredentials: (opts.allowCredentials || []).map(c => ({ ...c, id: b64urlToBuf(c.id) }));
  };
}

// 로그인 결과 인코딩
export function encodeAssertionCredential(cred) {
  return {
    id: cred.id,
    rawId: bufToB64url(cred.rawId),
    type: cred.type,
    response: {
      clientDataJSON: bufToB64url(cred.response.clientDataJSON),
      authenticatorData: bufToB64url(cred.response.authenticatorData),
      signature: bufToB64url(cred.response.signature),
      userHandle: cred.response.userHandle ? bufToB64url(cred.response.userHandle) : null
    },
    clientExtensionResults: cred.getClientExtensionResults()
  };
}

```

envelope 위치 주의: 위 `decodeCreationOptions(opts)` 의 `opts` 는 응답 **envelope** 전체가 아니라 **`data.publicKeyCredentialCreationOptions` (등록) / `data.publicKeyCredentialRequestOptions` (인증) 안쪽 객체**입니다. envelope을 먼저 풀어 그 객체만 꺼내 넘기세요. (\$5의 `postJson` 은 envelope을 풀어 `data` 를 반환하므로 `start.publicKeyCredentialCreationOptions` 로 접근합니다.)

도메인·HTTPS 주의: `rpId` 는 클라이언트가 접속하는 RP 서버 도메인과 일치해야 하며(예시는 `dev-passkey.crosscert.com`), 운영에서는 HTTPS가 필수입니다. 로컬 개발 시에는 `localhost` 가 안전한 컨텍스트로 예외 인정되어 HTTP로도 동작합니다(`localhost` 와 `127.0.0.1` 은 별개 origin이니 한쪽으로 통일하세요).

모바일 앱: 네이티브 앱은 위 JS 대신 OS API를 씁니다(상세는 § 7). Android는 Credential Manager가 **WebAuthn JSON**을 그대로 처리하므로 변환이 불필요하고, iOS는 AuthenticationServices에서 `challenge / userID` 를 `Data` 로 디코딩해 넘긴 뒤 결과를 base64url 인코딩해야 합니다.

4. 엔드포인트 레퍼런스

4.1 POST /passkey/register/begin

패스키 등록을 시작하고 creation options를 받습니다. RP 서버가 내부에서 `userHandle` 을 만들고 pending 상태를 세션에 저장합니다.

• 요청 body:

필드	타입	필수	설명
<code>username</code>	string	✓	사용자명입니다.
<code>displayName</code>	string	✓	authenticator에 표시될 이름입니다.

클라이언트는 이 **두 필드만** 보냅니다. `userHandle` (= `user.id`) 생성과 그 외 처리는 RP 서버가 수행하므로, 추가 필드를 보낼 필요가 없습니다.

• 응답 data (RegisterOptionsResp):

필드	타입	설명
<code>publicKeyCredentialCreationOptions</code>	object	WebAuthn creation options입니다. <code>decodeCreationOptions()</code> 로 변환해 <code>navigator.credentials.create()</code> 에 넘깁니다.

`publicKeyCredentialCreationOptions` 내부 필드입니다.

필드	타입	설명
<code>challenge</code>	string	서버 생성 challenge(base64url)입니다.
<code>rp.id / rp.name</code>	string	Relying Party ID와 표시 이름입니다.
<code>user.id</code>	string	RP 서버가 만든 <code>userHandle</code> (base64url)입니다. 클라이언트는 그대로 전달만 합니다.
<code>user.displayName / user.name</code>	string	요청의 <code>displayName / username</code> 입니다.
<code>pubKeyCredParams</code>	array	허용 알고리즘 <code>[ES256(-7), RS256(-257)]</code> 입니다.
<code>timeout</code>	number	ceremony 타임아웃(ms)입니다.
<code>attestation</code>	string	<code>none / indirect / direct / enterprise</code> 중 하나입니다.
<code>excludeCredentials</code>	array	이미 등록한 패스키 목록 <code>[{type:"public-key",id:<base64url>}]</code> 입니다(중복 등록 방지). <code>transports</code> 힌트는 의도적으로 포함하지 않습니다 (없어도 동작하며, 등록 시점 정보라 stale 위험이 있어 생략 — <code>allowCredentials</code> 도 동일).
<code>authenticatorSelection</code>	object	<code>userVerification</code> , <code>residentKey</code> 설정입니다.

• Request:

```
POST /passkey/register/begin
Content-Type: application/json
X-XSRF-TOKEN: <CSRF 토큰>

{ "username": "alice", "displayName": "Alice" }
```

● Response 200 OK :

```
{
  "success": true,
  "code": "OK",
  "message": "Success",
  "data": {
    "publicKeyCredentialCreationOptions": {
      "challenge": "k7Hd2...Qw",
      "rp": { "id": "dev-passkey.crosscert.com", "name": "Sample RP" },
      "user": { "id": "ZGV2LVZzZXItMDAx", "displayName": "Alice", "name": "alice" },
      "pubKeyCredParams": [ { "type": "public-key", "alg": -7 }, { "type": "public-key", "alg": -257 } ],
      "timeout": 60000,
      "attestation": "none",
      "excludeCredentials": [],
      "authenticatorSelection": { "userVerification": "preferred", "residentKey": "preferred" }
    }
  },
  "error": null,
  "traceId": "a1b2c3d4",
  "timestamp": "2026-06-01T12:00:00"
}
```

4.2 POST /passkey/register/finish

브라우저 등록 결과를 보내 검증·저장을 완료합니다. `begin` 과 같은 세션이어야 합니다(pending token). 세션이 만료되면 `W002` 가 나오므로, 그 경우 `begin` 부터 다시 시작하세요.

● 요청 body:

필드	타입	필수	설명
<code>publicKeyCredential</code>	object	✓	<code>navigator.credentials.create()</code> 결과를 <code>encodeAttestationCredential()</code> 로 인코딩한 것입니다.

● 응답 data (`RegistrationFinishResponse`):

필드	타입	설명
<code>credentialId</code>	string	등록된 credential ID(base64url)입니다.
<code>aaguid</code>	string	authenticator AAGUID입니다(메타데이터, 클라이언트가 직접 쓸 일은 거의 없습니다).
<code>attestationFormat</code>	string	attestation 포맷입니다(메타데이터).
<code>createdAt</code>	string(ISO instant)	생성 시작입니다.

● Request:

```
POST /passkey/register/finish
Content-Type: application/json
X-XSRF-TOKEN: <CSRF 토큰>

{
  "publicKeyCredential": {
    "id": "AbCd...Ef",
    "rawId": "AbCd...Ef",
    "type": "public-key",
    "response": {
      "clientDataJSON": "eyJ0eXB...",
      "attestationObject": "o2NmbXQ..."
    }
  },
  "clientExtensionResults": {}
}
```

● Response 200 OK :

```
{
  "success": true,
  "code": "OK",
  "message": "Passkey registered",
  "data": {
    "credentialId": "AbCd...Ef",
    "aaguid": "08987058-cadc-4b81-b6e1-30de50dcbe96",
    "attestationFormat": "none",
    "createdAt": "2026-06-01T12:00:01Z"
  },
  "error": null,
  "traceId": "a1b2c3d4",
  "timestamp": "2026-06-01T12:00:01"
}
```

4.3 POST /passkey/authenticate/begin

패스키 로그인을 시작하고 request options를 받습니다.

- 요청 body:

필드	타입	필수	설명
username	string	✕	지정하면 해당 사용자의 패스키로 줍힙니다. 생략하면 discoverable(usernameless) 로그인입니다.

- 응답 data (LoginOptionsResp):

필드	타입	설명
publicKeyCredentialRequestOptions	object	WebAuthn request options입니다. decodeRequestOptions() 로 변환해 navigator.credentials.get() 에 넘깁니다.

publicKeyCredentialRequestOptions 내부 필드입니다.

필드	타입	설명
challenge	string	서버 생성 challenge(base64url)입니다.
rpId	string	Relying Party ID입니다.
timeout	number	ceremony 타임아웃(ms)입니다.
userVerification	string	required 또는 preferred 입니다.
allowCredentials	array	username 이 있으면 그 사용자의 패스키 목록, 없으면 [] (discoverable)입니다.

- Request:

```
POST /passkey/authenticate/begin
Content-Type: application/json
X-XSRF-TOKEN: <CSRF 토큰>

{ "username": "alice" }
```

- Response 200 OK :


```
{
  "success": true,
  "code": "OK",
  "message": "Success",
  "data": {
    "publicKeyCredentialRequestOptions": {
      "challenge": "p9Lm3...Zx",
      "rpId": "dev-passkey.crosscert.com",
      "timeout": 60000,
      "userVerification": "preferred",
      "allowCredentials": [ { "type": "public-key", "id": "AbCd...Ef" } ]
    }
  },
  "error": null,
  "traceId": "b2c3d4e5",
  "timestamp": "2026-06-01T12:05:00"
}
```

4.4 POST /passkey/authenticate/finish

브라우저 로그인 결과를 보냅니다. RP 서버가 ID Token을 검증하고 **세션을 확립**합니다.

클라이언트 입장: `finish` 가 `200` 이면 로그인 성공이며 세션 쿠키로 인증됩니다. "누가 로그인했는가"의 판정은 RP 서버 몫이라 클라이언트가 신경 쓸 필요 없습니다.

RP 서버 구현자용 — discoverable 로그인에서는 username이 없으므로, RP 서버는 검증된 ID Token의 `sub` (= 등록 시 부여한 `userHandle`) 로 사용자를 역매핑합니다. 요청 body의 `response.userHandle` 은 검증 전 클라이언트 입력이므로 신뢰하지 말고 반드시 ID Token의 `sub` 를 쓰세요.

● 요청 body:

필드	타입	필수	설명
<code>publicKeyCredential</code>	object	✓	<code>navigator.credentials.get()</code> 결과를 <code>encodeAssertionCredential()</code> 로 인코딩한 것입니다.

- **응답 `data`:** `null` (성공 시 세션이 확립됩니다. 이후 요청은 세션 쿠키로 인증됩니다.)

● Request:

```
POST /passkey/authenticate/finish
Content-Type: application/json
X-XSRF-TOKEN: <CSRF 토큰>

{
  "publicKeyCredential": {
    "id": "AbCd...Ef",
    "rawId": "AbCd...Ef",
    "type": "public-key",
    "response": {
      "clientDataJSON": "eyJ0eXB...",
      "authenticatorData": "SZYN5V...",
      "signature": "MEUCIQ...",
      "userHandle": "ZGV2LXVzZXItMDAx"
    }
  },
  "clientExtensionResults": {}
}
```

- **Response `200 OK`:**

```
{
  "success": true,
  "code": "OK",
  "message": "Success",
  "data": null,
  "error": null,
  "traceId": "b2c3d4e5",
  "timestamp": "2026-06-01T12:05:02"
}
```

4.5 POST /logout

확립된 세션을 무효화합니다. Spring Security 기본 로그아웃이므로 envelope이 아닌 **302 리다이렉트**(`Location: /`)로 응답합니다.

- **요청 body:** 없음. CSRF 토큰(`X-XSRF-TOKEN`)이 필요합니다.
- **응답:** `302 Found` , `Location: /` . 서버 세션이 무효화되고 세션 쿠키가 만료됩니다.
- **Request:**

```
POST /logout
X-XSRF-TOKEN: <CSRF 토큰>
```

- **Response** `302 Found` :

```
HTTP/1.1 302 Found
Location: /
Set-Cookie: JSESSIONID=...; Max-Age=0
```

fetch로 호출할 때 리다이렉트를 따라가지 않으려면 `redirect: 'manual'` 을 쓰고, 응답 후 클라이언트가 보관한 세션 쿠키를 폐기하세요.

5. 전체 흐름 (브라우저 JS)

sample-rp의 실제 등록·로그인 흐름입니다.

등록

```
import { decodeCreationOptions, encodeAttestationCredential, postJson } from './js/helpers.js';

const start = await postJson('/passkey/register/begin', { username, displayName });
const cred = await navigator.credentials.create({
  publicKey: decodeCreationOptions(start.publicKeyCredentialCreationOptions)
});
const fin = await postJson('/passkey/register/finish', {
  publicKeyCredential: encodeAttestationCredential(cred)
});
// fin.credentialId 등록 완료
```

로그인

```
import { decodeRequestOptions, encodeAssertionCredential, postJson } from '/js/helpers.js';

const start = await postJson('/passkey/authenticate/begin', { username }); // username 생략 가능
const assertion = await navigator.credentials.get({
  publicKey: decodeRequestOptions(start.publicKeyCredentialRequestOptions)
});
await postJson('/passkey/authenticate/finish', {
  publicKeyCredential: encodeAssertionCredential(assertion)
});
window.location = '/'; // 세션 확립됨
```

postJson 은 CSRF 토큰을 붙이고 envelope을 풀어 data 를 반환하며, 실패 시 code / message / traceId 를 담은 에러를 던집니다. 아래 구현은 Thymeleaf <meta> 전용입니다(순수 SPA는 § 2 참조).

```
export async function postJson(url, body) {
  const csrf = document.querySelector('meta[name=csrf]').content ?? '';
  const res = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'X-XSRF-TOKEN': csrf },
    body: JSON.stringify(body),
    credentials: 'same-origin'
  });
  const env = await res.json();
  if (!env.success) {
    const err = new Error(env.message || 'Unknown error');
    err.code = env.code; err.traceId = env.traceId; err.fieldErrors = env.error?.fieldErrors;
    throw err;
  }
  return env.data;
}
```

6. 에러 코드

실패 응답은 § 2 envelope의 code 로 식별합니다.

code	HTTP	의미	상황
C001	400	INVALID_INPUT	필드 검증 실패(예: username / displayName 누락), 잘못된 입력입니다.
C002	405	METHOD_NOT_ALLOWED	잘못된 HTTP 메서드입니다.
C003	404	ENTITY_NOT_FOUND	리소스가 없습니다.
C999	500	INTERNAL_SERVER_ERROR	서버 오류입니다.
A001	401	UNAUTHORIZED	인증이 필요합니다.
W001	409	USERNAME_TAKEN	이미 등록된 username입니다.
W002	400	PENDING_REG_MISSING	세션에 진행 중인 등록이 없습니다(begin 없이 finish 호출, 또는 세션 만료).
W003	400	PENDING_AUTH_MISSING	세션에 진행 중인 로그인인 없습니다(begin 없이 finish 호출, 또는 세션 만료). 이 경우 authenticate/begin 부터 다시 시작하세요.
P001	502	PASSKEY_API_ERROR	RP 서버가 호출한 Passkey 서버에서 오류가 발생했습니다.
P003	429	PASSKEY_RATE_LIMITED	Passkey 서버 rate limit을 초과했습니다. 클라이언트는 즉시 재시도하지 말고 지수 백오프(예: 1s→2s→4s)로 재시도하세요. 응답에 Retry-After 헤더가 있으면 그 값을 우선합니다.
P004	401	PASSKEY_ID_TOKEN	ID Token 검증에 실패했습니다(인증 ceremony 검증 실패, 또는 iss / aud 불일치 포함). RP 서버가 ID Token의 iss / aud (tenantId 기반)를 검증할 때 tenantId는 UUID 소문자 대시 형식(7f00dead-0000-...)으로 옵니다. RP 설정값을 RAW hex로 두면 표기 차이로 mismatch가 나므로, 비교 전 UUID로 정규화하거나 설정을 UUID 형식으로 맞추세요.

P00x 코드는 RP 서버가 Passkey 서버와 통신하면서 만나는 오류입니다. 클라이언트 입장에서는 "서버 측 일시 오류"로 처리하고 재시도하면 됩니다.

에러 응답 예시

세션 만료로 등록 완료 실패 (400):

```
{
  "success": false,
  "code": "W002",
  "message": "No pending registration in session",
  "data": null,
  "error": { "errorCode": "W002", "fieldErrors": null },
  "traceId": "e5f6a7b8",
  "timestamp": "2026-06-01T12:03:00"
}
```

필드 검증 실패 (400):

```
{
  "success": false,
  "code": "C001",
  "message": "Invalid input value",
  "data": null,
  "error": {
    "errorCode": "C001",
    "fieldErrors": [ { "field": "username", "rejectedValue": null, "reason": "must not be blank" } ]
  },
  "traceId": "e5f6a7b8",
  "timestamp": "2026-06-01T12:00:00"
}
```

7. 모바일 네이티브 앱 연동

네이티브 앱은 `navigator.credentials` 대신 OS 패스키 API를 씁니다. **RP 서버 엔드포인트(§ 4)와 요청/응답 형식은 동일하며**, 달라지는 것은 (1) ceremony 수행 주체가 OS API라는 점, (2) 세션·CSRF를 앱이 직접 관리한다는 점입니다.

7.1 공통 — 플랫폼 요구사항·세션·CSRF·도메인 연결

- **플랫폼 최소 버전 / 의존성:**
- **iOS:** AuthenticationServices 패스키 API(`ASAuthorizationPlatformPublicKeyCredentialProvider`)는 **iOS 16+**, conditional UI(autofill) 등 일부는 17+입니다. 본 예시는 iOS 16 이상을 가정합니다.
- **Android:** `androidx.credentials 1.2.0 이상`. 구형 기기 호환을 위해 play-services 백엔드도 함께 추가합니다. `gradle implementation "androidx.credentials:credentials:1.2.0" implementation "androidx.credentials:credentials-play-services-auth:1.2.0"`
- **세션 쿠키:** 네이티브 앱에는 브라우저 쿠키 jar가 없으므로 HTTP 클라이언트가 쿠키를 보관·재전송하도록 설정해야 합니다. iOS는 `URLSession` 이 `HTTPCookieStorage` 로 자동 처리하고, Android는 OkHttp에 `CookieJar` 를 설정합니다. **기본 CookieJar는 메모리 전용**이라 앱 재시작 시 세션이 초기화되니, 유지가 필요하면 `PersistentCookieJar` 같은 디스크 저장 구현을 쓰세요. `begin` → `finish` 요청이 같은 세션 쿠키를 공유해야 합니다(§ 1).
- **CSRF 토큰(§ 2):** 첫 GET으로 `XSRF-TOKEN` 쿠키를 받아두고 이후 POST마다 `X-XSRF-TOKEN` 헤더에 씁니다. 쿠키 jar가 쿠키 저장은 자동으로 하지만 **헤더로 옮겨 쓰는 것은 앱이 직접** 해야 합니다(7.1.1 코드). 토큰은 세션과 함께 유지되므로 보통 앱 시작 시 1회 GET이면 충분하나, `finish` 에서 `W002` / `W003` 가 나면 세션 만료이니 GET부터 다시 합니다.
- **도메인 연결(필수):** 네이티브 패스키는 앱과 도메인의 소유 관계를 OS가 검증합니다. 이 설정이 없으면 OS가 ceremony를 거부합니다.
- **iOS:** Associated Domains에 `webcredentials:<rpId>` 추가 + 서버의 `https://<rpId>/.well-known/apple-app-site-association` 에 앱 App ID 등록.
- **Android:** Digital Asset Links — 서버의 `https://<rpId>/.well-known/assetlinks.json` 에 앱 패키지명·서명 지문 등록.
- 여기서 `<rpId>` 는 `options` 응답의 `rp.id` (등록) / `rpId` (인증)와 일치해야 합니다.

7.1.1 envelope 풀기 · CSRF 헤더 (양 플랫폼 공통 개념)

RP 응답은 **envelope**으로 감싸여 옵니다. OS API에 넘길 때는 **envelope**을 풀어 **data** 안쪽만 써야 합니다.

단계	iOS에서 쓰는 값	Android에서 쓰는 값
등록 begin 응답	<code>data.publicKeyCredentialCreationOptions</code> 안의 <code>rp.id</code> · <code>challenge</code> · <code>user.id</code> · <code>user.name</code>	<code>data.publicKeyCredentialCreationOptions</code> 객체 전체를 JSON 문자열로
인증 begin 응답	<code>data.publicKeyCredentialRequestOptions</code> 안의 <code>rpId</code> · <code>challenge</code>	<code>data.publicKeyCredentialRequestOptions</code> 객체 전체를 JSON 문자열로

등록 응답은 `rp.id` (중첩), 인증 응답은 `rpId` (평면)로 키가 다릅니다(§ 4.1 vs § 4.3). 혼동하지 마세요.

POST 요청에 CSRF 토큰을 싣는 코드입니다.

```
// iOS — HTTPCookieStorage 에서 XSRF-TOKEN 을 꺼내 헤더에 싣고 POST
func postJson(_ urlString: String, _ body: [String: Any]) async throws -> [String: Any] {
    let url = URL(string: urlString)!
    var req = URLRequest(url: url)
    req.httpMethod = "POST"
    req.setValue("application/json", forHTTPHeaderField: "Content-Type")
    if let cookies = HTTPCookieStorage.shared.cookies(for: url),
        let xsrf = cookies.first(where: { $0.name == "XSRF-TOKEN" }) {
        req.setValue(xsrf.value, forHTTPHeaderField: "X-XSRF-TOKEN")
    }
    req.httpBody = try JSONSerialization.data(withJSONObject: body)
    let (data, _) = try await URLSession.shared.data(for: req) // URLSession 이 세션 쿠키 자동 첨부
    return try JSONSerialization.jsonObject(with: data) as! [String: Any]
}
```

```
// Android — OkHttp CookieJar 가 본 XSRF-TOKEN 을 보관했다가 헤더로 옮긴다
class XsrfCookieJar : CookieJar {
    @Volatile private var xsrf: String? = null
    private val store = mutableMapOf<String, List<Cookie>>() // 세션 쿠키 보관(데모 수준)
    override fun saveFromResponse(url: HttpUrl, cookies: List<Cookie>) {
        store[url.host] = cookies
        cookies.find { it.name == "XSRF-TOKEN" }?.let { xsrf = it.value }
    }
    override fun loadForRequest(url: HttpUrl): List<Cookie> = store[url.host] ?: emptyList()
    fun token(): String? = xsrf
}

val jar = XsrfCookieJar()
val client = OkHttpClient.Builder().cookieJar(jar).build()
// 첫 GET 으로 XSRF-TOKEN 확보 후, POST 마다:
val req = Request.Builder().url(url)
    .header("X-XSRF-TOKEN", jar.token() ?: "")
    .post(body.toRequestBody("application/json".toMediaType()))
    .build()
```

7.2 iOS (AuthenticationServices)

iOS는 base64url 문자열을 **Data** 로 디코딩해 OS API에 넘기고, 결과로 받은 **Data** 필드들을 다시 **base64url**로 인코딩해 RP 서버에 보냅니다(웹의 § 3 변환과 같은 역할).

표준 **Data** 는 base64url을 직접 다루지 못하므로(`base64Encoded` 만 있음), 아래 확장을 먼저 둡니다. § 3의 `b64urlToBuf` / `bufToB64url` 과 같은 변환입니다.

```

extension Data {
    init?(base64URLEncoded s: String) {
        var t = s.replacingOccurrences(of: "-", with: "+")
            .replacingOccurrences(of: "_", with: "/")
        t.append(String(repeating: "=", count: (4 - t.count % 4) % 4)) // 패딩 복원
        self.init(base64Encoded: t)
    }
    func base64URLEncodedString() -> String {
        base64EncodedString()
            .replacingOccurrences(of: "+", with: "-")
            .replacingOccurrences(of: "/", with: "_")
            .replacingOccurrences(of: "=", with: "")
    }
}

```

등록 — `/passkey/register/begin` 응답의 `challenge` · `user.id` 를 `Data` 로(아래 `opts` 는 § 7.1.1대로 envelope을 푼 `data.publicKeyCredentialCreationOptions`):

```

import AuthenticationServices

let provider = ASAuthorizationPlatformPublicKeyCredentialProvider(
    relyingPartyIdentifier: opts.rp.id // rp.id

let request = provider.createCredentialRegistrationRequest(
    challenge: Data(base64URLEncoded: opts.challenge)!, // base64url → Data
    name:      opts.user.name,
    userID:    Data(base64URLEncoded: opts.user.id)!) // userHandle

let controller = ASAuthorizationController(authorizationRequests: [request])
controller.delegate = self
controller.presentationContextProvider = self // 필수 — 없으면 런타임 크래시
controller.performRequests()

// delegate — 결과를 base64url로 인코딩해 /passkey/register/finish 로 전송
func authorizationController(controller: ASAuthorizationController,
    didCompleteWithAuthorization auth: ASAuthorization) {
    let c = auth.credential as! ASAuthorizationPlatformPublicKeyCredentialRegistration
    let body: [String: Any] = ["publicKeyCredential": [
        "id":      c.credentialID.base64URLEncodedString(),
        "rawID":    c.credentialID.base64URLEncodedString(),
        "type":     "public-key",
        "response": [
            "clientDataJSON": c.rawClientDataJSON.base64URLEncodedString(),
            "attestationObject": c.rawAttestationObject!.base64URLEncodedString()
        ],
        "clientExtensionResults": [:]
    ],
    ]
    // POST /passkey/register/finish (X-XSRF-TOKEN, 세션 쿠키 포함)
}

```

인증 — `/passkey/authenticate/begin` 응답의 `challenge` 를 `Data` 로:

```

let provider = ASAuthorizationPlatformPublicKeyCredentialProvider(
    relyingPartyIdentifier: opts.rpId) // rpId

let request = provider.createCredentialAssertionRequest(
    challenge: Data(base64URLEncoded: opts.challenge!))

let controller = ASAuthorizationController(authorizationRequests: [request])
controller.delegate = self
controller.presentationContextProvider = self // 필수 — 없으면 런타임 크래시
controller.performRequests()

// delegate — assertion 결과를 base64url 인코딩해 /passkey/authenticate/finish 로
func authorizationController(controller: ASAuthorizationController,
    didCompleteWithAuthorization auth: ASAuthorization) {
    let c = auth.credential as! ASAuthorizationPlatformPublicKeyCredentialAssertion
    let body: [String: Any] = ["publicKeyCredential": [
        "id": c.credentialID.base64URLEncodedString(),
        "rawId": c.credentialID.base64URLEncodedString(),
        "type": "public-key",
        "response": [
            "clientDataJSON": c.rawClientDataJSON.base64URLEncodedString(),
            "authenticatorData": c.rawAuthenticatorData.base64URLEncodedString(),
            "signature": c.signature.base64URLEncodedString(),
            "userHandle": c.userID.base64URLEncodedString()
        ],
        "clientExtensionResults": [:]
    ]]
    // POST /passkey/authenticate/finish
}

```

`controller.presentationContextProvider = self` 에 더해, 클래스에 `ASAuthorizationControllerPresentationContextProviding` 를 채택하고 `presentationAnchor(for:)` 에서 현재 윈도우를 반환해야 OS 모달이 뜹니다(미구현 시 크래시). 에러는 별도 delegate 메서드로 오며 § 8.2에서 다룹니다.

7.3 Android (Credential Manager)

Android의 Credential Manager는 **WebAuthn 표준 JSON을 그대로** 주고받으므로 base64url 변환이 불필요합니다. 단, RP 응답은 envelope으로 감싸여 오므로 `requestJson` 에 넘길 때는 **envelope을 풀어 data 안쪽 options 객체만** JSON 문자열로 만들어야 합니다. envelope째 넘기면 Credential Manager가 표준 필드를 못 찾아 `CreateCredentialException` 으로 실패합니다.

등록 — `data.publicKeyCredentialCreationOptions` 만 떼어 `requestJson` 으로:

```

import androidx.credentials.CredentialManager
import androidx.credentials.CreatePublicKeyCredentialRequest
import androidx.credentials.CreatePublicKeyCredentialResponse
import androidx.credentials.GetCredentialRequest
import androidx.credentials.GetPublicKeyCredentialOption
import androidx.credentials.PublicKeyCredential
import org.json.JSONObject

val credentialManager = CredentialManager.create(context)

// 1) begin 응답(envelope)에서 options 객체만 추출 → 문자열화
val beginEnvelope = JSONObject(beginResponseBody) // RP begin 응답 전체
val optionsJson = beginEnvelope.getJSONObject("data")
    .getJSONObject("publicKeyCredentialCreationOptions")
    .toString() // ← 이 문자열만 넘긴다

// 2) OS 패스키 생성
val request = CreatePublicKeyCredentialRequest(requestJson = optionsJson)
val result = credentialManager.createCredential(context, request)
    as CreatePublicKeyCredentialResponse

// 3) 결과 JSON 을 publicKeyCredential 로 한 번 감싸 finish 로 (변환 없음)
val body = """{ "publicKeyCredential": ${result.registrationResponseJson} }"""
// POST /passkey/register/finish (X-XSRF-TOKEN, 세션 쿠키 포함 — § 7.1.1)

```

인증 — `data.publicKeyCredentialRequestOptions` 만 떼어 `requestJson` 으로:

```

val optionsJson = JSONObject(beginResponseBody).getJSONObject("data")
    .getJSONObject("publicKeyCredentialRequestOptions")
    .toString()

val option = GetPublicKeyCredentialOption(requestJson = optionsJson)
val request = GetCredentialRequest(ListOf(option))
val result = credentialManager.getCredential(context, request)

val cred = result.credential as PublicKeyCredential
val body = """{ "publicKeyCredential": ${cred.authenticationResponseJson} }"""
// POST /passkey/authenticate/finish

```

`registrationResponseJson` / `authenticationResponseJson` 은 이미 § 4의 `publicKeyCredential` 객체와 같은 형식(WebAuthn 표준 JSON)입니다. RP 서버 요청 body는 이를 `publicKeyCredential` 키로 한 번 감싼 형태이므로 위처럼 감싸 보내면 됩니다.

8. 클라이언트 측 에러 처리

§ 6 에러 코드는 RP 서버가 내려주는 오류입니다. 그 외에 OS·브라우저가 ceremony 수행 중에 던지는 오류가 있으며, 이는 서버 응답이 아니라 `navigator.credentials.*` (웹) 또는 OS delegate/exception(앱)에서 발생합니다. 사용자에게 보여줄 메시지를 다르게 처리해야 합니다.

8.1 웹 (`navigator.credentials`)

`create()` / `get()` 는 실패 시 `DOMException` 을 던집니다. `err.name` 으로 분기합니다.

err.name	상황	권장 처리
<code>NotAllowedError</code>	사용자가 취소했거나 ceremony 타임아웃입니다(돌을 구분할 수 없음).	"취소되었거나 시간이 초과되었습니다. 다시 시도해 주세요." — 재시도 버튼 노출. 에러로 시끄럽게 알리지 마세요(정상적인 취소 포함).
<code>InvalidStateError</code>	등록 시 이미 등록된 authenticator입니다(<code>excludeCredentials</code> 충돌).	"이미 이 기기에 패스키가 등록되어 있습니다." — 로그인으로 안내.
<code>NotSupportedError</code>	요청한 알고리즘/옵션을 authenticator가 지원하지 않습니다.	"이 기기에서 지원하지 않는 패스키입니다."
<code>SecurityError</code>	<code>rpId</code> 가 현재 origin과 불일치하거나 안전하지 않은 컨텍스트입니다(HTTPS/localhost 아님).	개발/설정 오류입니다. <code>rpId</code> · origin · HTTPS를 점검하세요.
<code>AbortError</code>	<code>AbortSignal</code> 로 중단되었습니다.	보통 무시(앱이 의도적으로 취소한 경우).

```

try {
  const cred = await navigator.credentials.get({ publicKey: decodeRequestOptions(opts) });
  // ...
} catch (e) {
  if (e.name === 'NotAllowedError') {
    showRetry('취소되었거나 시간이 초과되었습니다. ');
  } else if (e.name === 'InvalidStateError') {
    showInfo('이미 등록된 기기입니다. ');
  } else {
    showError(`패스키 오류: ${e.name}`);
  }
}

```

"패스키 없음"(authenticator에 자격증명이 없음) 은 별도 에러로 오지 않고, 인증 ceremony에서 사용자가 선택할 패스키가 없어 결과적으로 `NotAllowedError` (취소/타임아웃)로 귀결됩니다. 따라서 "패스키 없음"을 명시적으로 구분할 수는 없으며, 재시도/등록 안내로 처리합니다.

8.2 모바일 네이티브

iOS — 실패는 ceremony delegate의 별도 메서드로 옵니다. `ASAuthorizationError` 로 캐스팅해 `code` 로 분기합니다.

```
func authorizationController(controller: ASAuthorizationController,
    didCompleteWithError error: Error) {
    guard let e = error as? ASAuthorizationError else { showError("알 수 없는 오류"); return }
    switch e.code {
    case .canceled:                break // 사용자 취소 — 조용히(알림 금지)
    case .failed, .invalidResponse: showRetry("인증에 실패했습니다. 다시 시도해 주세요.")
    case .notHandled, .unknown:    showError("오류가 발생했습니다.")
    @unknown default:             showError("지원하지 않는 오류입니다.")
    }
}
```

Android — `createCredential` 은 `CreateCredentialException` , `getCredential` 은 `GetCredentialException` 을 던집니다. import 경로는 `androidx.credentials.exceptions.*` 입니다.

```
import androidx.credentials.exceptions.GetCredentialException
import androidx.credentials.exceptions.GetCredentialCancellationException
import androidx.credentials.exceptions.GetCredentialInterruptedException
import androidx.credentials.exceptions.NoCredentialException
// 등록 측: androidx.credentials.exceptions.CreateCredentialException,
//          androidx.credentials.exceptions.CreateCredentialCancellationException

try {
    val result = credentialManager.getCredential(context, request)
    // ...
} catch (e: GetCredentialException) {
    when (e) {
        is GetCredentialCancellationException -> { /* 사용자 취소 — 조용히 처리 */ }
        is NoCredentialException              -> { /* 선택할 패스키 없음 → 등록 안내 */ }
        is GetCredentialInterruptedException -> { /* 일시 중단 — 재시도 가능 */ }
        else                                  -> { /* Unknown/ProviderConfiguration/Unsupported 등 — 일반 오류 */ }
    }
}
```

- `NoCredentialException` (인증 시)은 선택할 패스키가 없는 경우로, 웹과 달리 Android는 이를 명시적으로 구분할 수 있어 등록으로 안내하기 좋습니다.

8.3 서버 오류와의 구분

발생 위치	형태	예
OS/브라우저 (ceremony)	<code>DOMException</code> (웹) / OS 예외(앱)	사용자 취소, 타임아웃, 미지원 기기
RP 서버	\$ 6 envelope <code>code</code>	<code>W002</code> (세션 만료), <code>C001</code> (입력 오류), <code>P004</code> (ID Token 검증 실패)

ceremony 단계(OS/브라우저)와 `finish` 응답(RP 서버)을 각각 `try/catch` 로 감싸 둘을 구분해 메시지를 다르게 보여주는 것을 권장합니다. 예: `begin` / `finish` 는 서버 오류(\$ 6)로, 그 사이 `navigator.credentials.*` 는 OS 오류(\$ 8.1)로 처리합니다.